adders

Half adder (2 bit)

$S = a \oplus b$

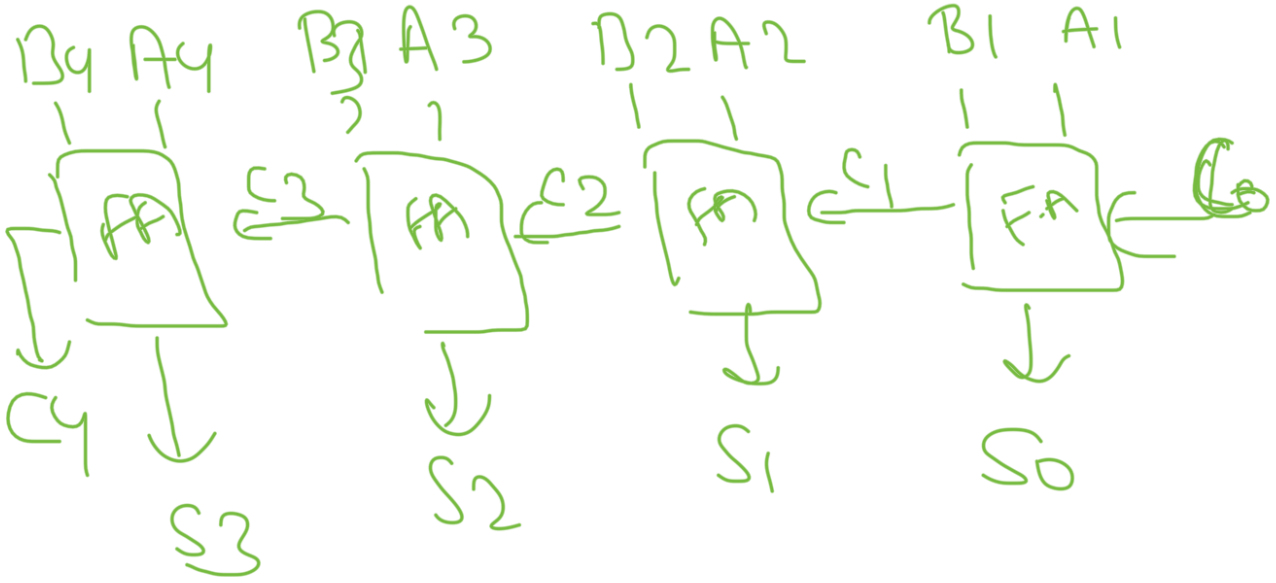
$C = ab$

full adder (3 bit)

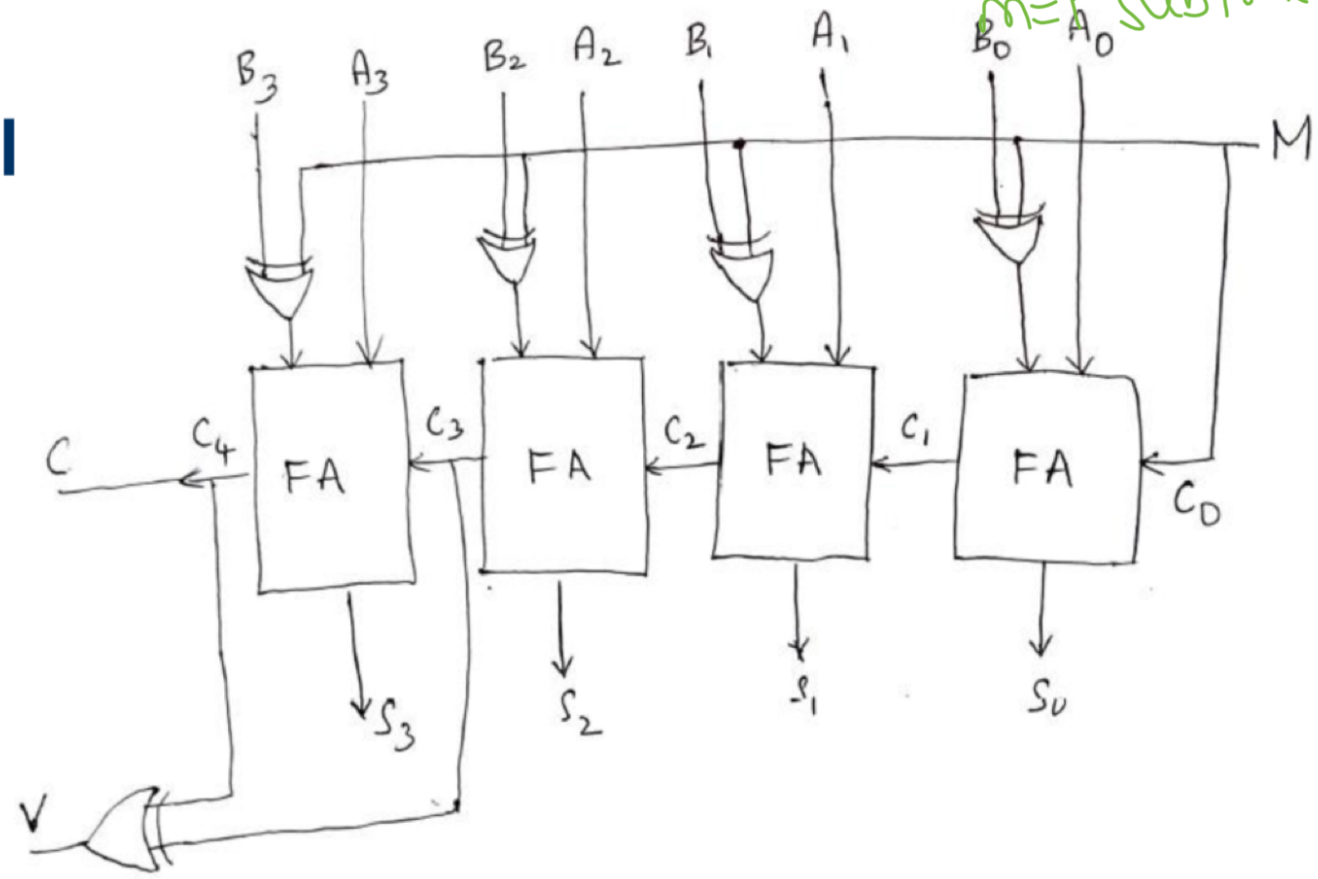
$S = a \oplus b \oplus c$

$c = ab + bc + ca$

Low hardware → ripple carry



m=0 adder
m=1 subtracter



drawback: time taking

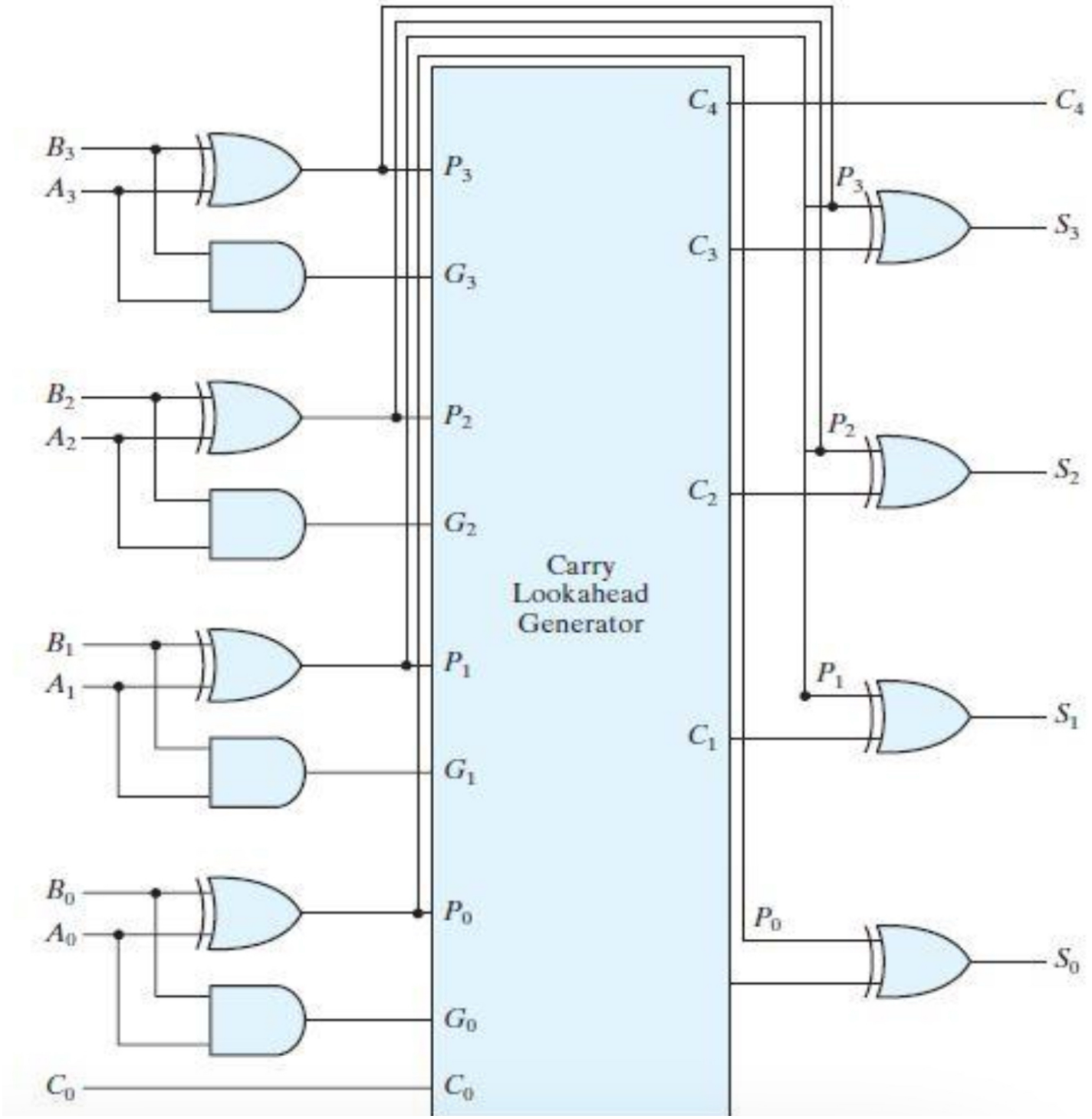
unsigned ⇒ C detects carry for add^n / borrow for sub^n

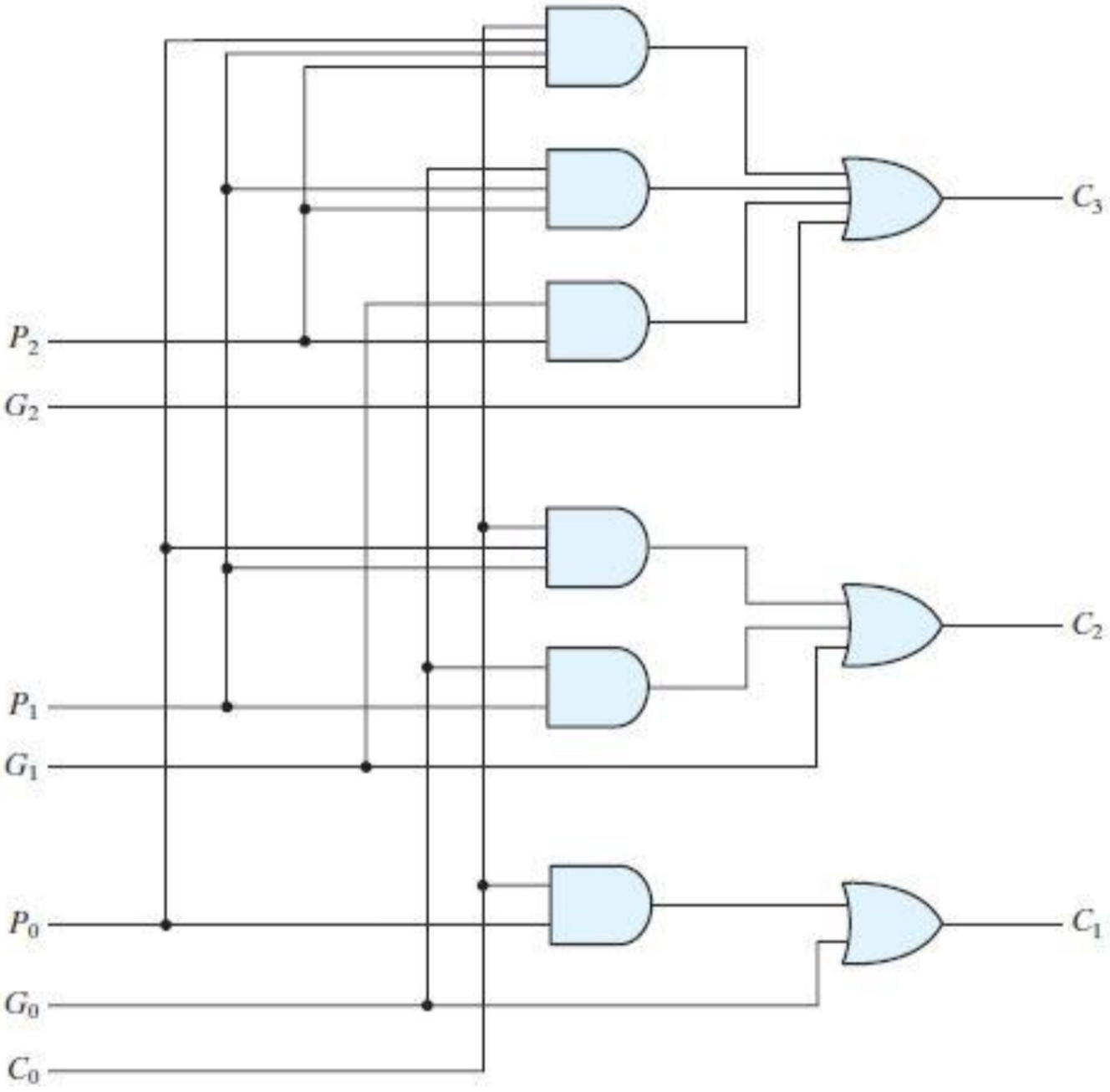
signed ⇒ V=0 → no overflow occured
V=1 → overflow occured

---

carry lookahead adder

$G_i = A_iB_i$ $P_i = A_i \oplus B_i$


B_3
A_3
B_2
A_2
B_1
A_1
B_0
A_0
C_0
P_3
G_3
P_2
G_2
P_1
G_1
P_0
G_0
C_0
Carry
Lookahead
Generator
C_4
C_3
C_2
C_1
C_4
P_3
S_3
P_2
S_2
P_1
S_1
P_0
S_0


P_2
G_2
P_1
G_1
P_0
G_0
C_0
C_3
C_2
C_1

$C_i = G_{i-1} + P_{i-1} C_{i-1}$ $S_i = P_i \oplus C_i$

more hardware $= A_i \oplus B_i \oplus C_i$

fan-in: number of inputs to a gate. high fan-in leads to lesser performance

decoder: $n \to 2^n$

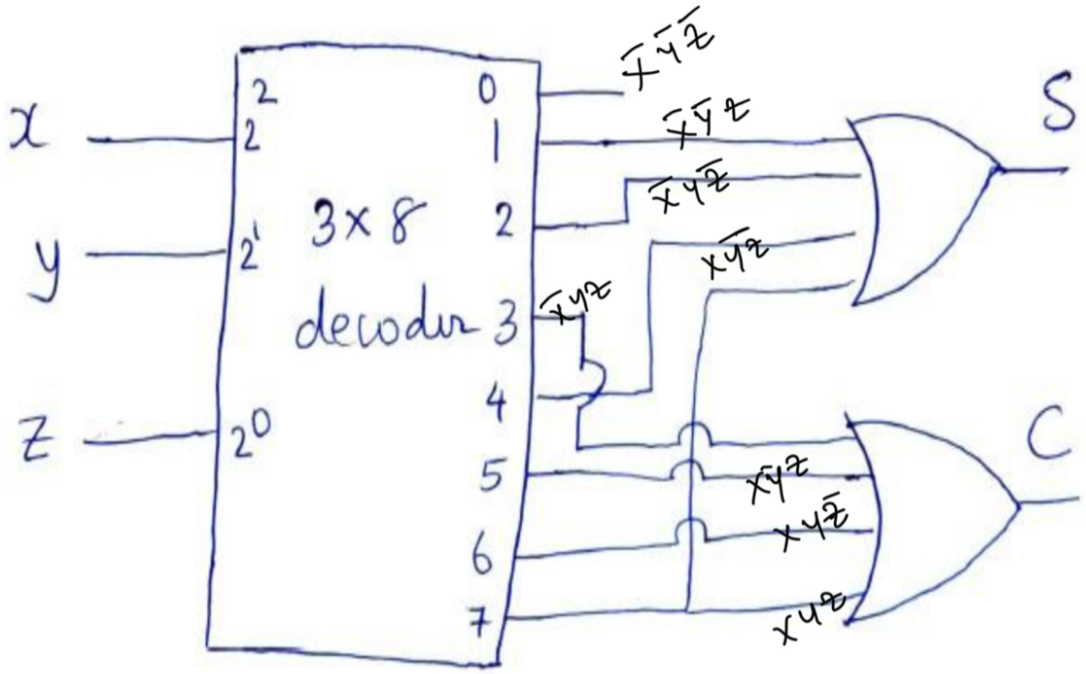
$D_0 = \bar{x}\bar{y}$ $D_1 = \bar{x}y$

$D_2 = x\bar{y}$ $D_3 = xy$

x	y	D_0	D_1	D_2	D_3
0	0	1	0	0	0
0	1	0	1	0	0

1 0 0 0 1

1 1 0 0 0 1



encoder : $2^n \to n$

Inputs								Outputs		
D_0	D_1	D_2	D_3	D_4	D_5	D_6	D_7	x	y	z
1	0	0	0	0	0	0	0	0	0	0
0	1	0	0	0	0	0	0	0	0	1
0	0	1	0	0	0	0	0	0	1	0
0	0	0	1	0	0	0	0	0	1	1
0	0	0	0	1	0	0	0	1	0	0
0	0	0	0	0	1	0	0	1	0	1
0	0	0	0	0	0	1	0	1	1	0
0	0	0	0	0	0	0	1	1	1	1

$z = D_1 + D_3 + D_5 + D_7$

$y = D_2 + D_3 + D_6 + D_7$

$x = D_4 + D_5 + D_6 + D_7$

priority encoder:

Inputs				Outputs		
D_0	D_1	D_2	D_3	x	y	V
0	0	0	0	X	X	0
1	0	0	0	0	0	1
X	1	0	0	0	1	1
X	X	1	0	1	0	1
X	X	X	1	1	1	1

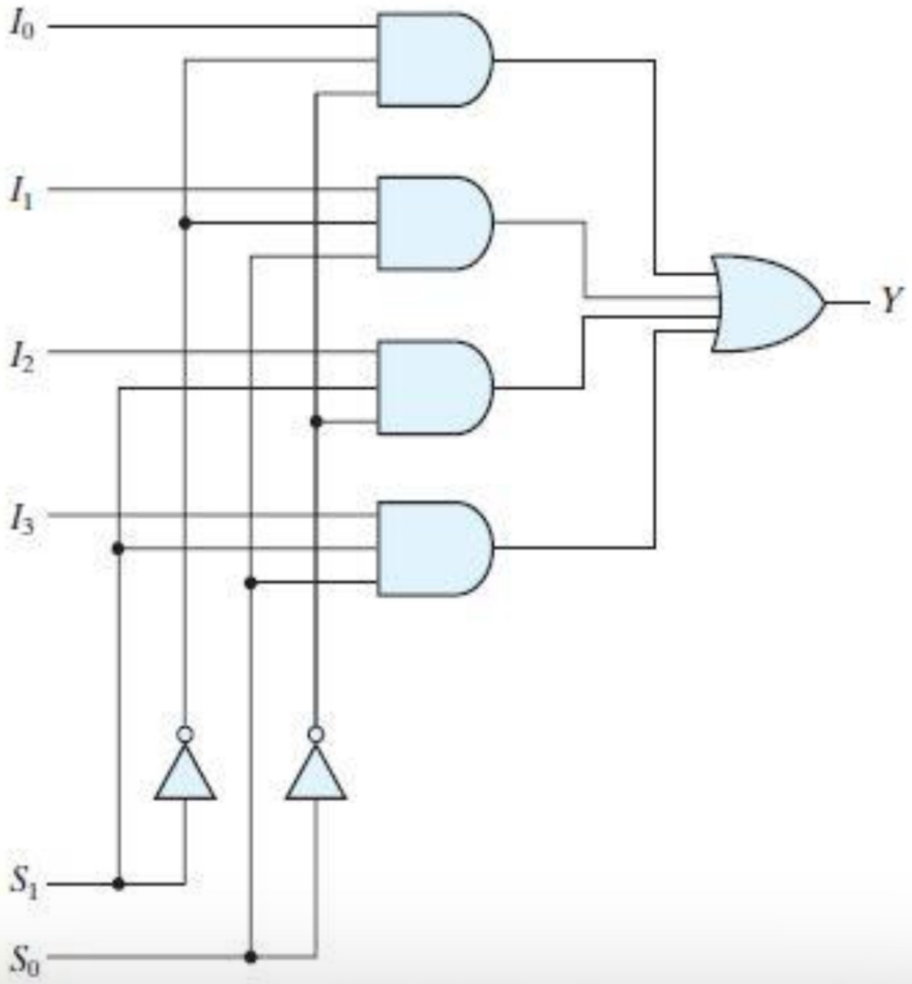
$x = D_2 + D_3$

$y = D_3 + \overline{D_2} D_1$

$V = D_0 + D_1 + D_2 + D_3$

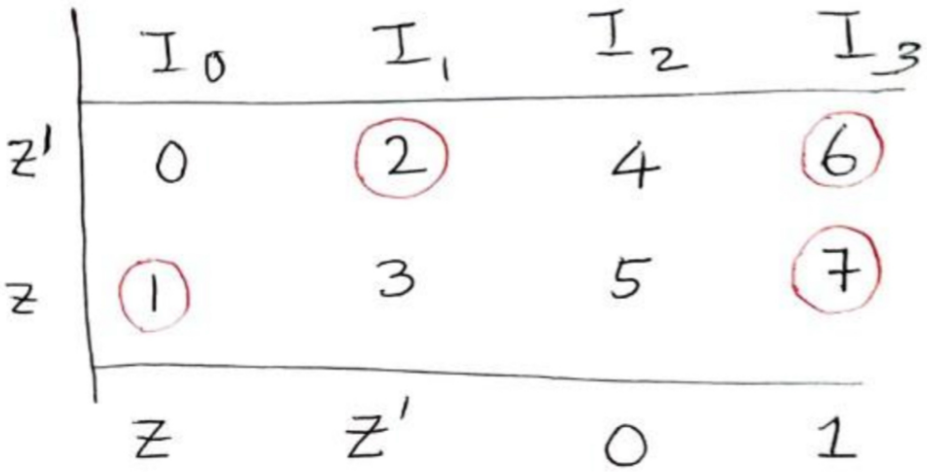
---

multiplexer: selects binary info from one of many inputs and directs it to single output

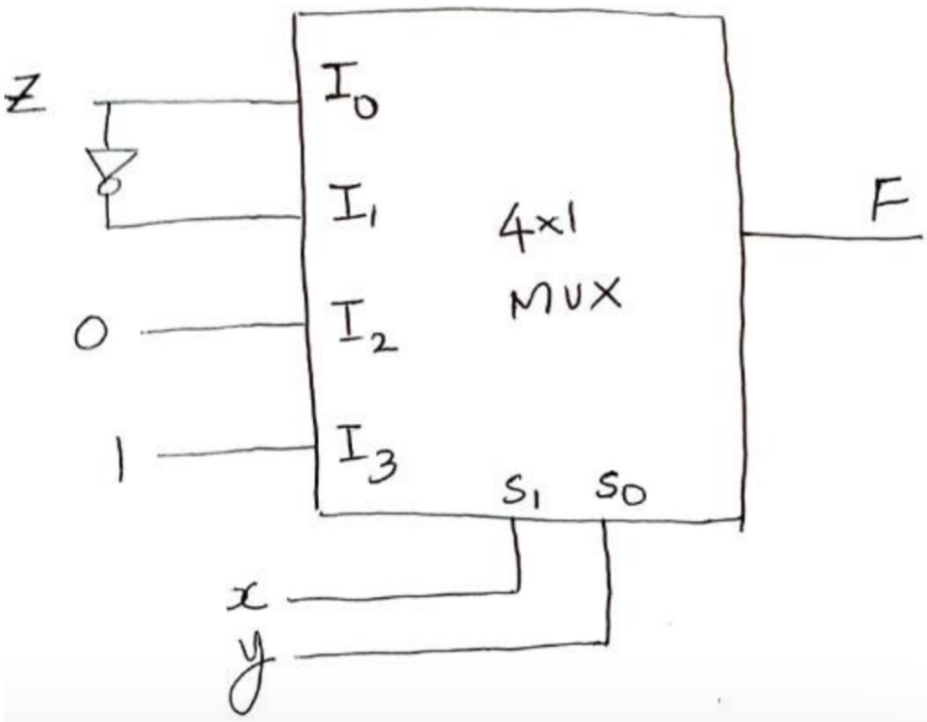


S_1	S_0	Y
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3

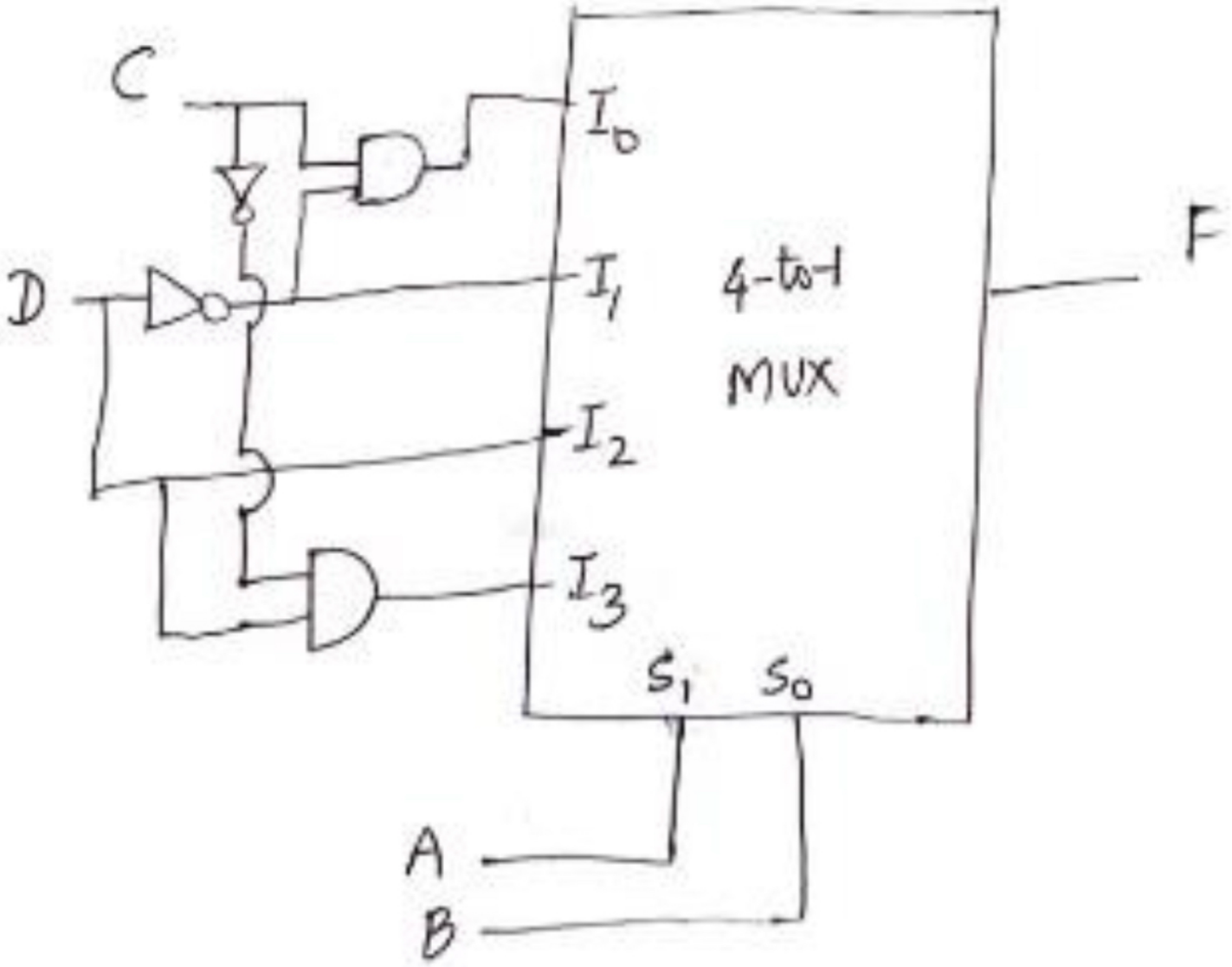
Implementation table for F (x, y, z) = ∑ (1,2,6,7)



Mux implementation for F (x, y, z) = ∑ (1,2,6,7)

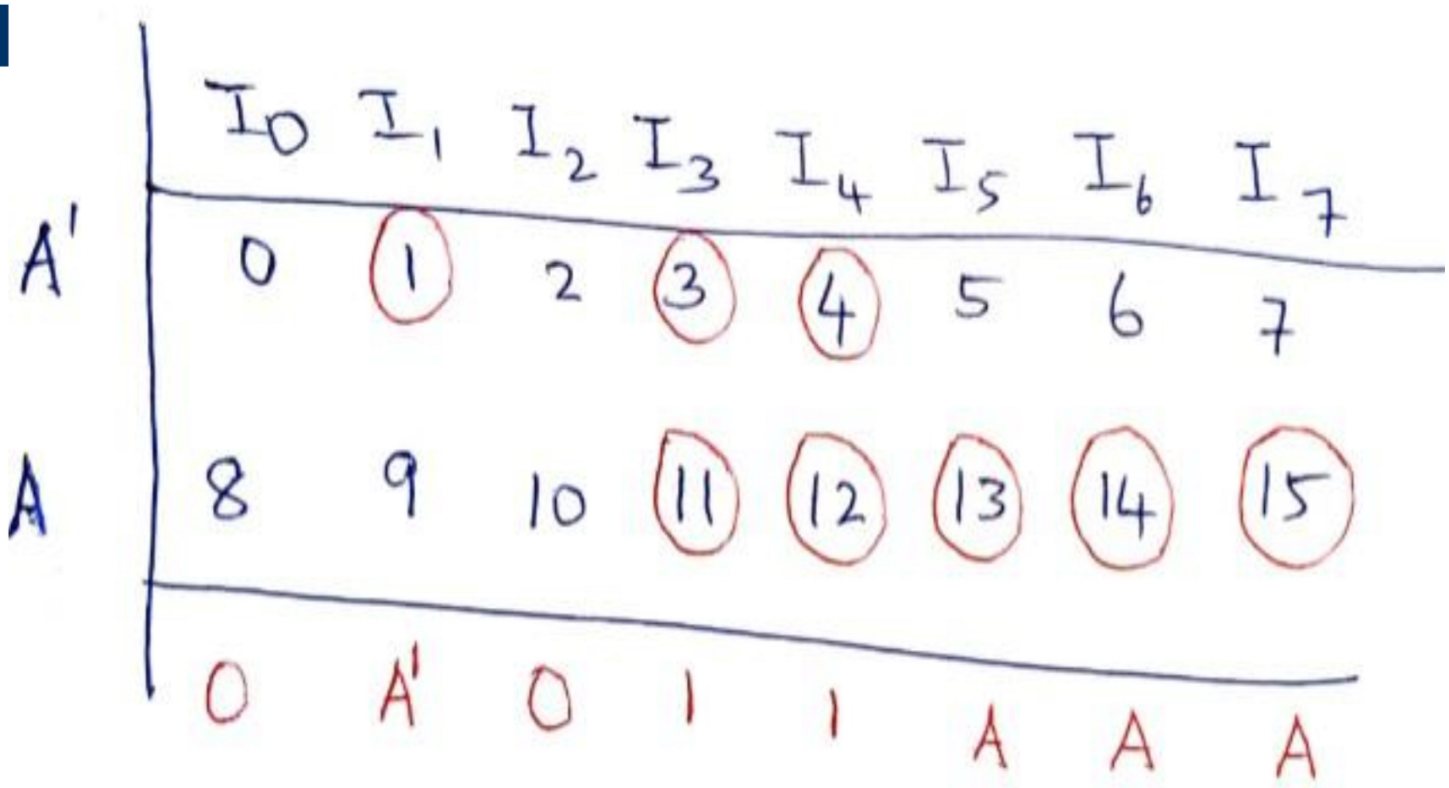


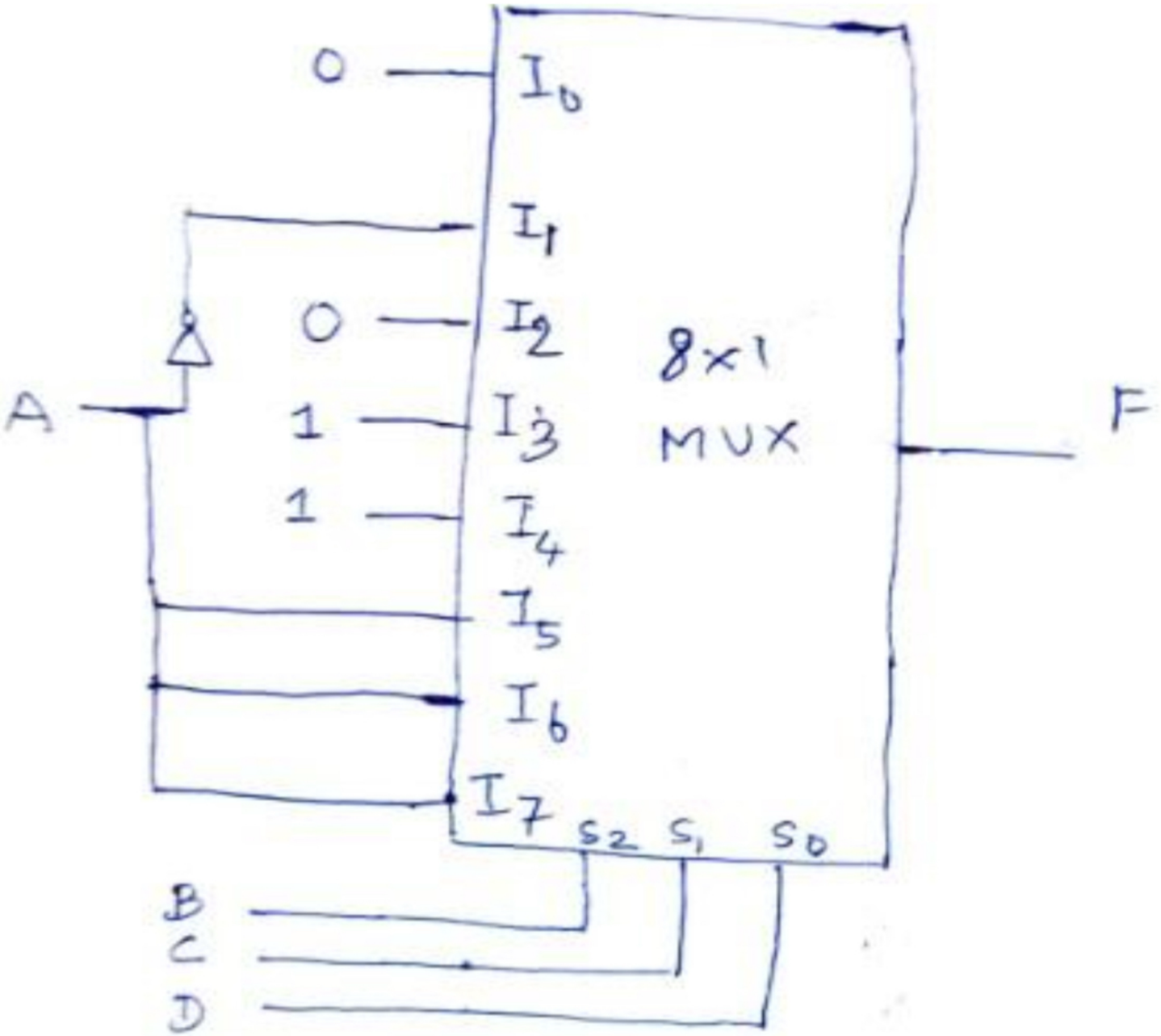
AB (S_1S_0)	I_0 (00)	I_1 (01)	I_2 (10)	I_3 (11)
$\bar{C}\bar{D}$ (00)	0	(4)	8	12
$\bar{C}D$ (01)	1	5	(9)	(13)
$C\bar{D}$ (10)	(2)	(6)	10	14
CD (11)	3	7	(11)	15
	$C\bar{D}$	$\bar{D}$	D	$\bar{C}D$


C
D
I_0
I_1
I_2
I_3
4-to-1
MUX
F
S_1
S_0
A
B

Realizing F(A,B,C,D) =∑ (1,3,4,11,12,13,14,15)

- The function here has 4 variables. We can use (n-1) variables for the selection input. So 3 selection lines and an 8-to-1 mux.
- We will let first variable B to be connected to input S_2 and the other variables (C and D) to be connected to S_1 and S_0 respectively.
- Data inputs (A, A', 0, 1) will come from the implementation table.

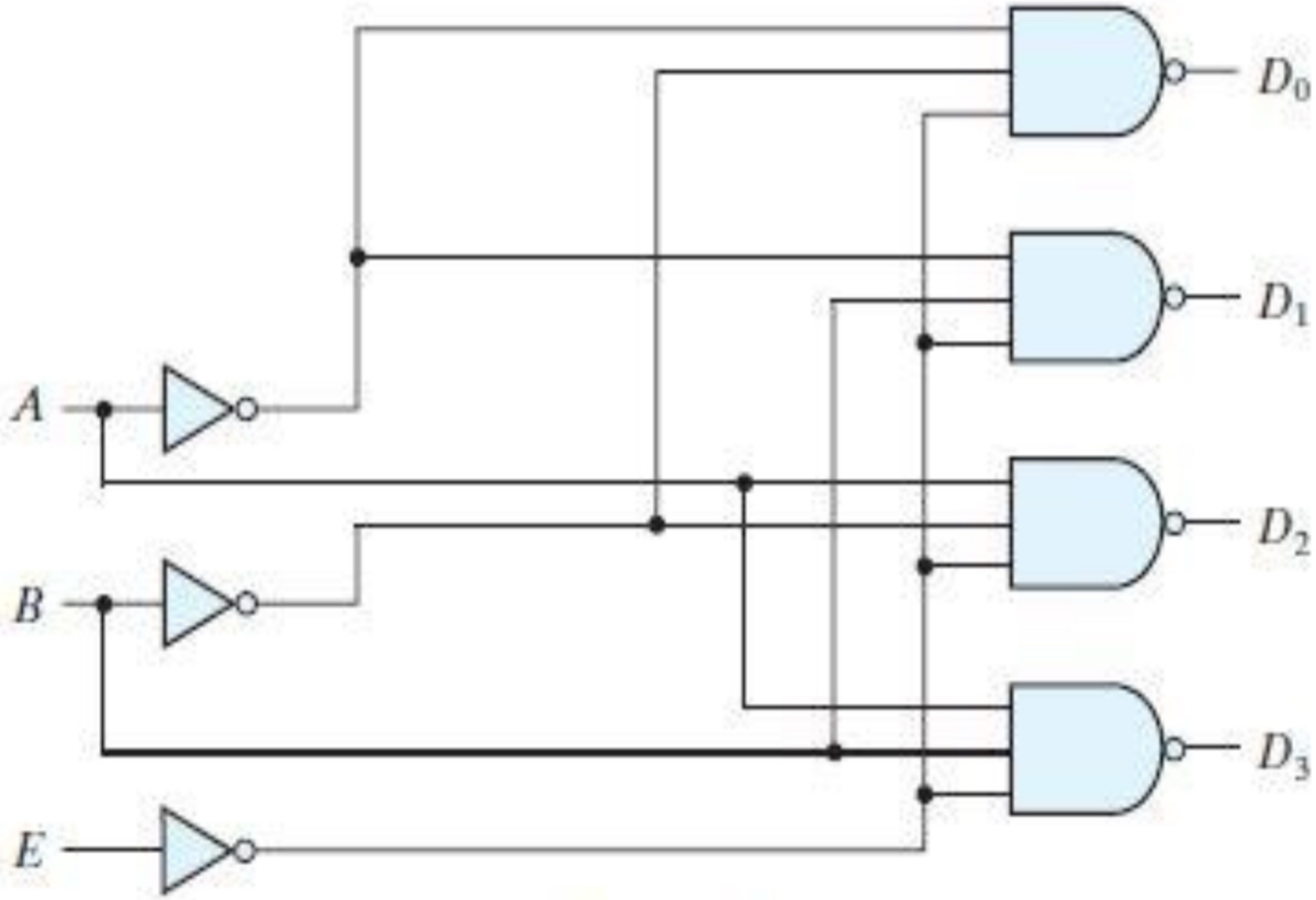




P

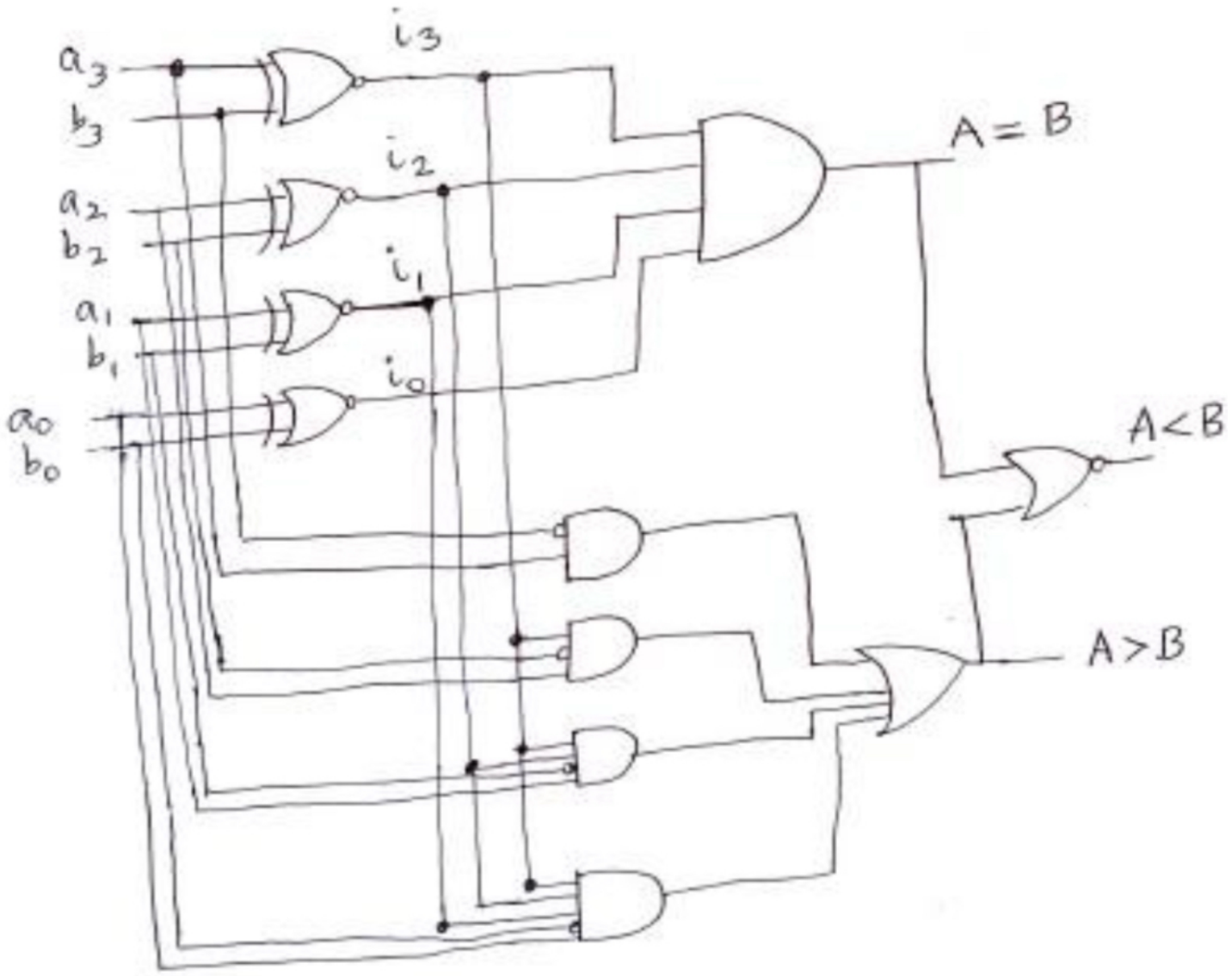
---

demultiplexer :



comparator:

1 bit: XOR gate


a_3
b_3
i_3
a_2
b_2
i_2
a_1
b_1
i_1
a_0
b_0
i_0
A = B
A < B
A > B

parity: when digital information is transmitted, some bits may be corrupted.

so we make sure:

even 1's in even parity

odd 1's in odd parity

p = even parity: $x_3 \oplus x_2 \oplus x_1 \oplus x_0$

c = receive: $p \oplus x_3 \oplus x_2 \oplus x_1 \oplus x_0$

$c \neq 0 \rightarrow$ error